Распределенная обработка графов, библиотека Galois

Мелашенко Ксения, Б-11

Введение

Цель эксперимента – оценить масштабируемость графовых алгоритмов по количеству узлов

Исследуемые алгоритмы:

- **BFS** (Breadth-first search)
- **TC** (Triangle Counting)
- **SSSP** (Single-Source Shortest Path)
- **PageRank**

Постановка эксперимента

1. **Собираем пары конфигураций:**
   N MPI-процессов (mpirun -n=N) и M потоков (-t=M).
   Суммарно не больше 8

2. **10 прогонов по каждой конфигурации**

3. **Для BFS и SSSP стартовая вершина выбирается случайно**

Оборудование

- **Операционная система**: MacOS Sequoia 15.6.1
- **Процессор**: Apple M2, 8 ядер (4 P-cores, 4 E-cores)
- **ОЗУ**: 8GB RAM
- **ПЗУ**: 256GB SSD
- **Версия Python**: 3.12.5
- **Open MPI:** 4.1.6
- **Clang:** 16.0.0
- **CMake:** 3.28
- **Galois:** v6.0.0 (master 13.05.26)
- **Партиционирование**: –partition=oec
- **Режим**: –exec=async

GALOIS_DO_NOT_BIND_THREADS! :(

Описание набора данных для экспериментов

Графы – SNAP Stanford

Для **TC** были выбраны **неориентированные** и **невзвешенные** графы:

- com-DBLP – 317 080 nodes, 1 049 866 edges
- com-Youtube – 1 134 890 nodes, 2 987 624 edges
- com-LiveJournal – 3 997 962 nodes, 34 681 189 edges

Описание набора данных для экспериментов

Графы – SNAP Stanford

Для **SSSP** были выбраны те же графы, что для **TC**, и добавлены случайные веса путем встроенной утилиты gr2randomweightgr библиотеки Galois:

(--gr2randomweightgr, --edgeType=int32, --minValue=1, --maxValue=100)

Описание набора данных для экспериментов

Графы – SNAP Stanford

Для **BFS** и **PageRank** были выбраны **ориентированные**, **невзвешенные** графы с одной компонентой связности:

- web-Google – 875 713 nodes, 5 105 039 edges
- wiki-Talk – 2 394 385 nodes, 5 021 410 edges
- wiki-topcats – 1 791 489 nodes, 28 511 807 edges

Статистика BFS, PageRank и SSSP

1. **Time** – время исполнения алгоритма. Берется медиана по всем прогонам
2. **Replication Factor** – среднее число копий вершин, хранящихся на разных узлах.
   При одном хосте (n=1) копий нет, поэтому всегда равен 1
3. **Sync** – время синхронизации между хостами

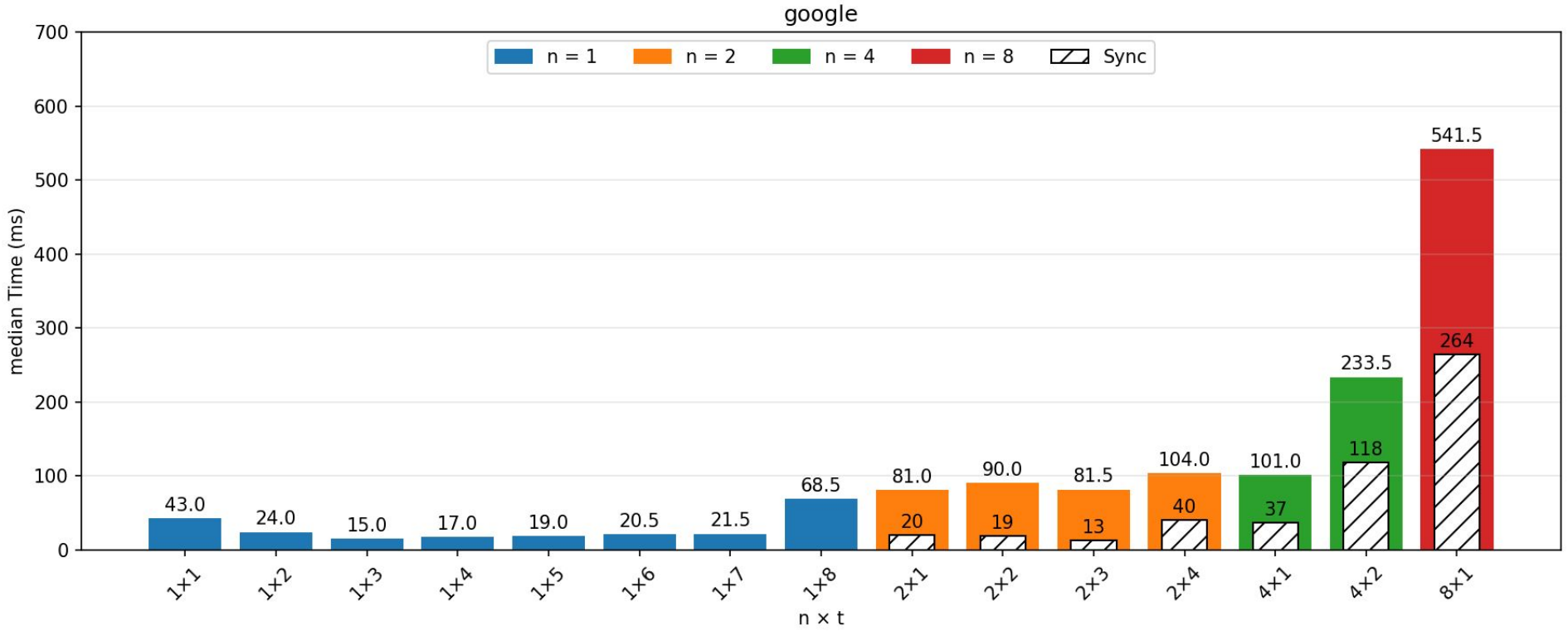
Статистика TC

1. **Time** – время исполнения алгоритма. Берется медиана по всем прогонам
2. **Replication Factor Nodes** – среднее число копий вершины
3. **Replication Factor Edges** – среднее число копий ребра.
   При одном хосте (n=1) копий нет, поэтому всегда равен 1, как и в случае с вершинами
4. **Graph Construct Time** – время построения графа

Результаты BFS

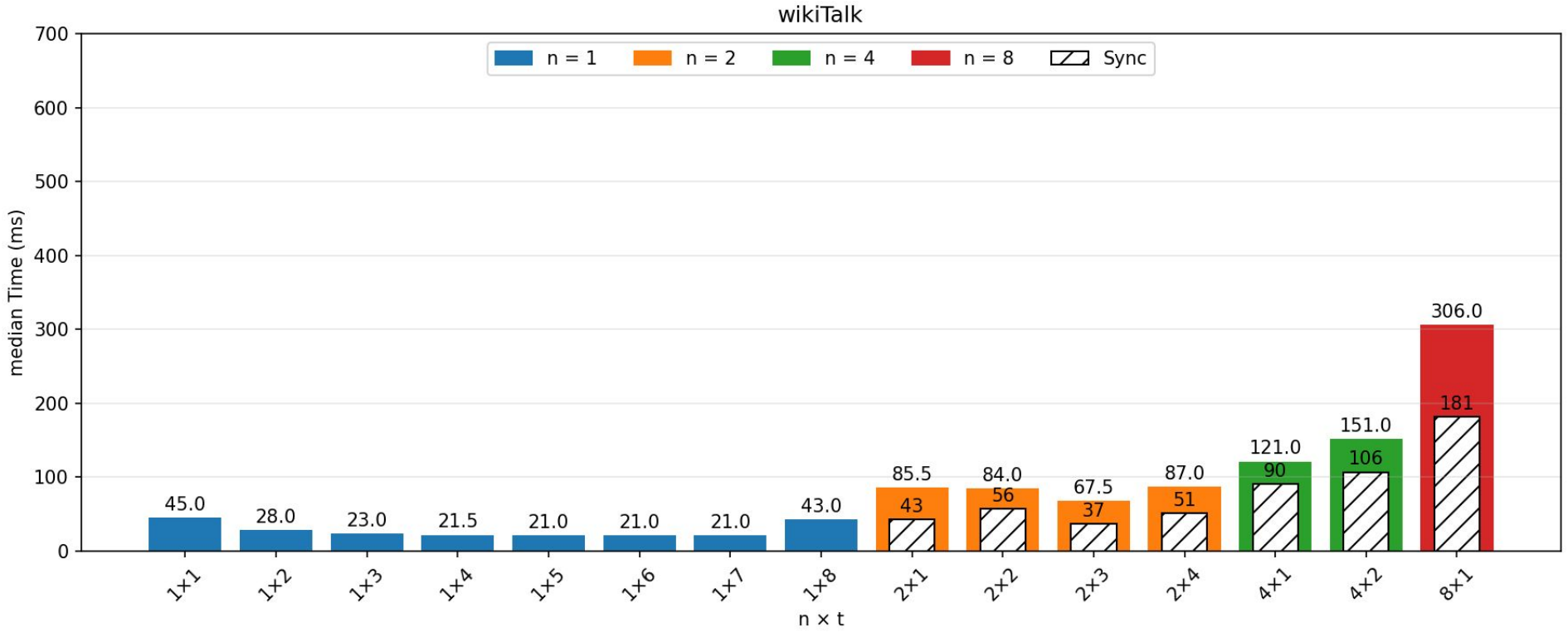
Время работы алгоритма BFS

web-Google



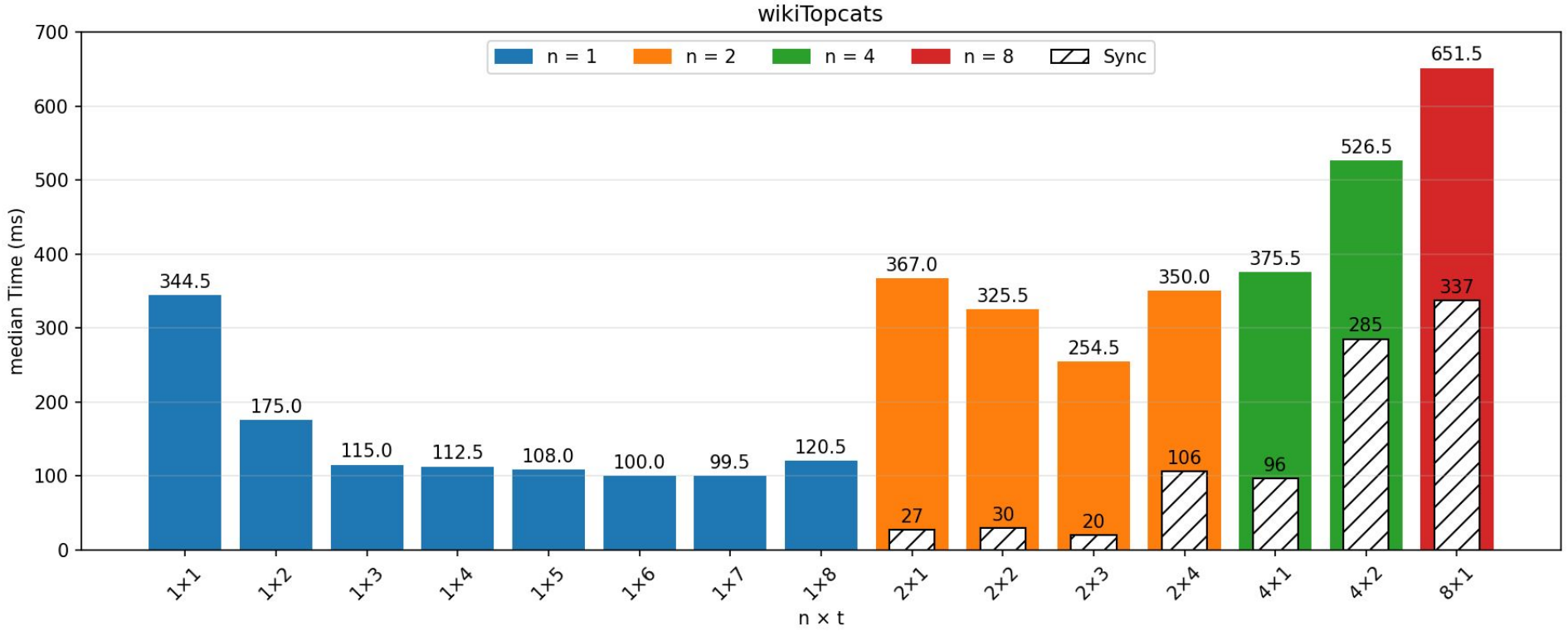
Время работы алгоритма BFS

wiki-Talk



Время работы алгоритма BFS

wiki-topcats



Итерации

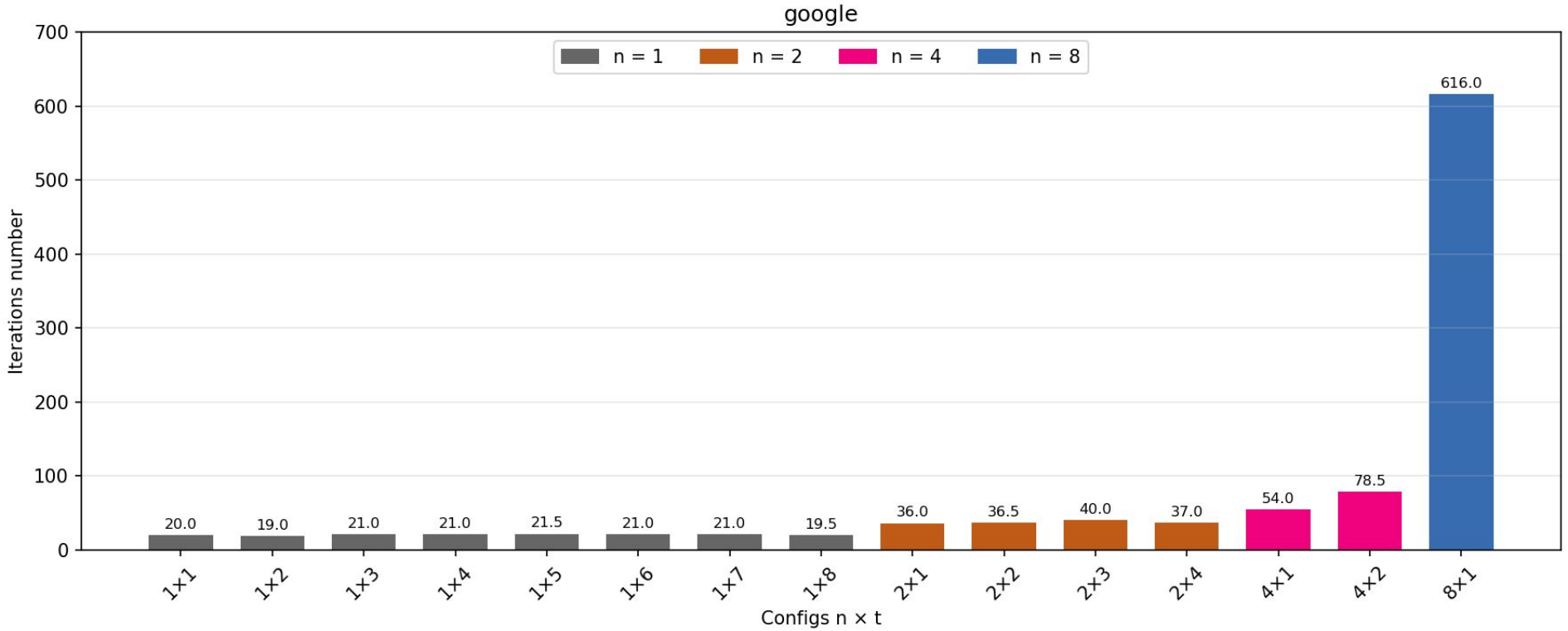
Итерация (раунд) – один проход цикла `do { do_all; sync; reduce } while (...)`

do_all: Хост перебирает все свои вершины, проверяя условие `dist_old > dist_current` (уменьшилось ли расстояние до вершины). Если да, то вершина называется «активной» и она передает новое значение соседям, иначе пропускается.

В асинхронном режиме , хост продолжает «крутить» **пустые раунды**, т.е его вершины не активны и никакой новой информации в sync для него не появилось.

Количество итераций BFS (максимум по хостам)

web-Google



Итерация (раунд) – один проход цикла `do { do_all; sync; reduce } while (...)`

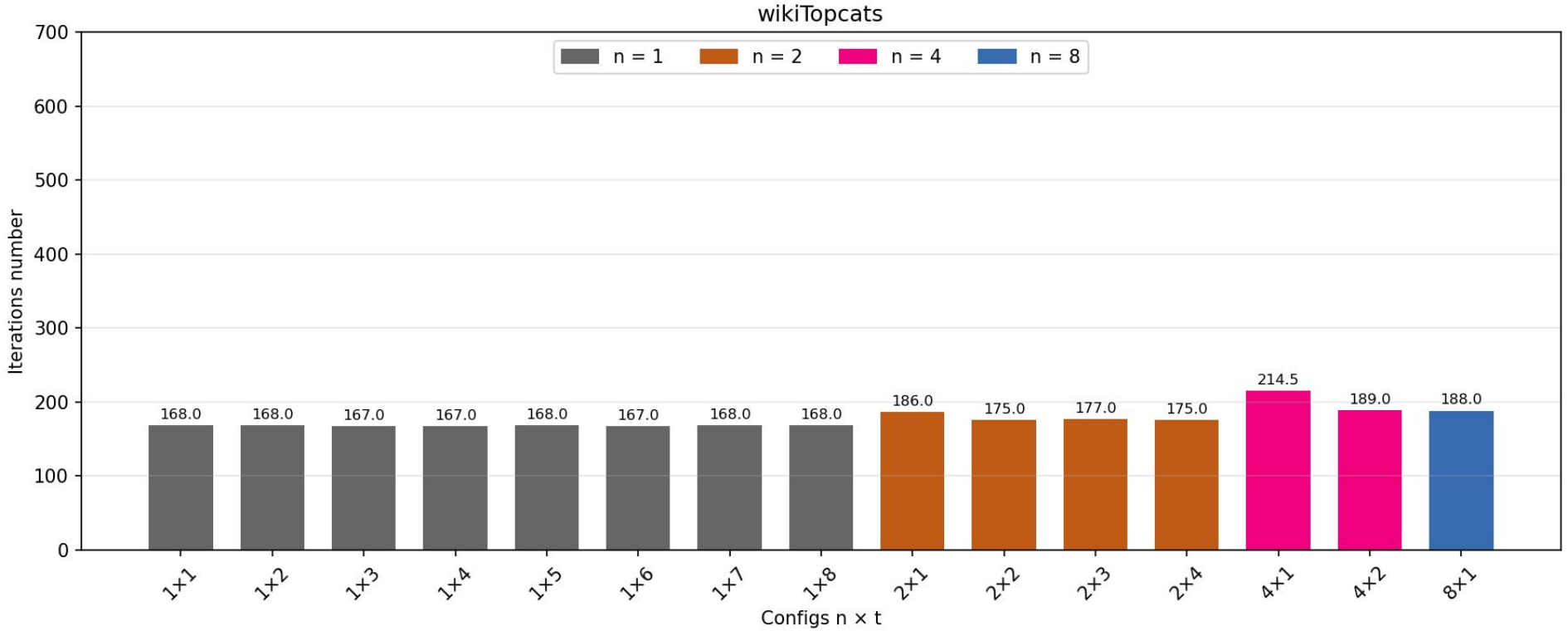
Проблема с округлением таймера Galois

Оставшееся время синхронизации не попадало в столбец Sync

- Galois округляет время в миллисекундах вниз до целых
- Каждая итерация теряет дробную часть миллисекунды
- Раунд короче 1 мс не учитывается (вместо этого 0 мс)

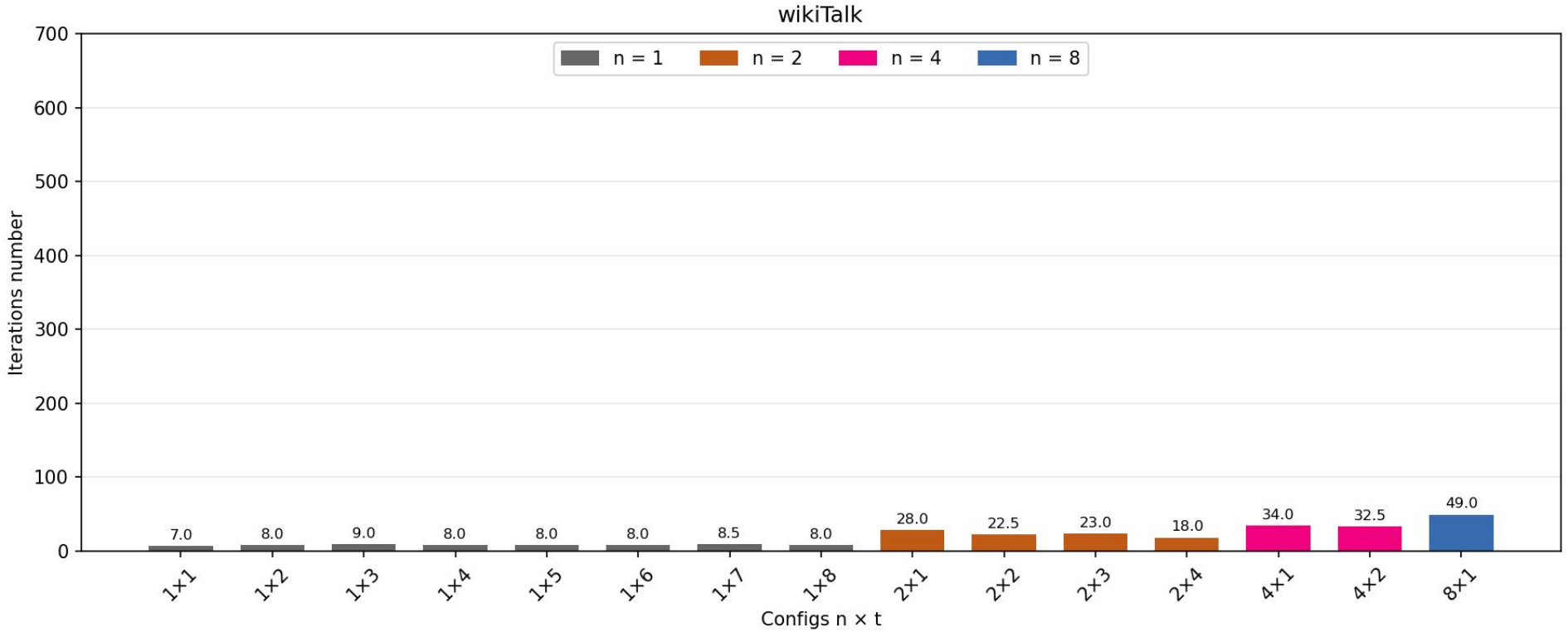
Количество итераций BFS (максимум по хостам)

wiki-topcats



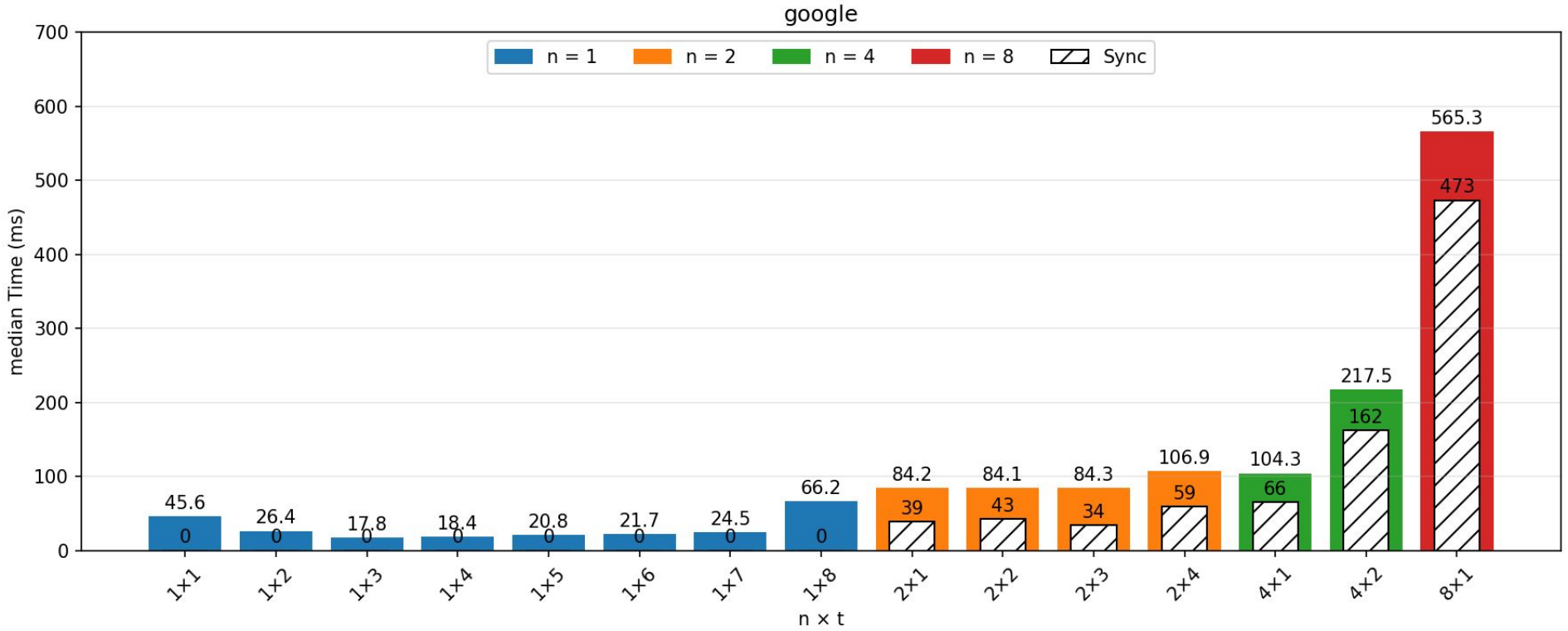
Количество итераций BFS (максимум по хостам)

wiki-Talk



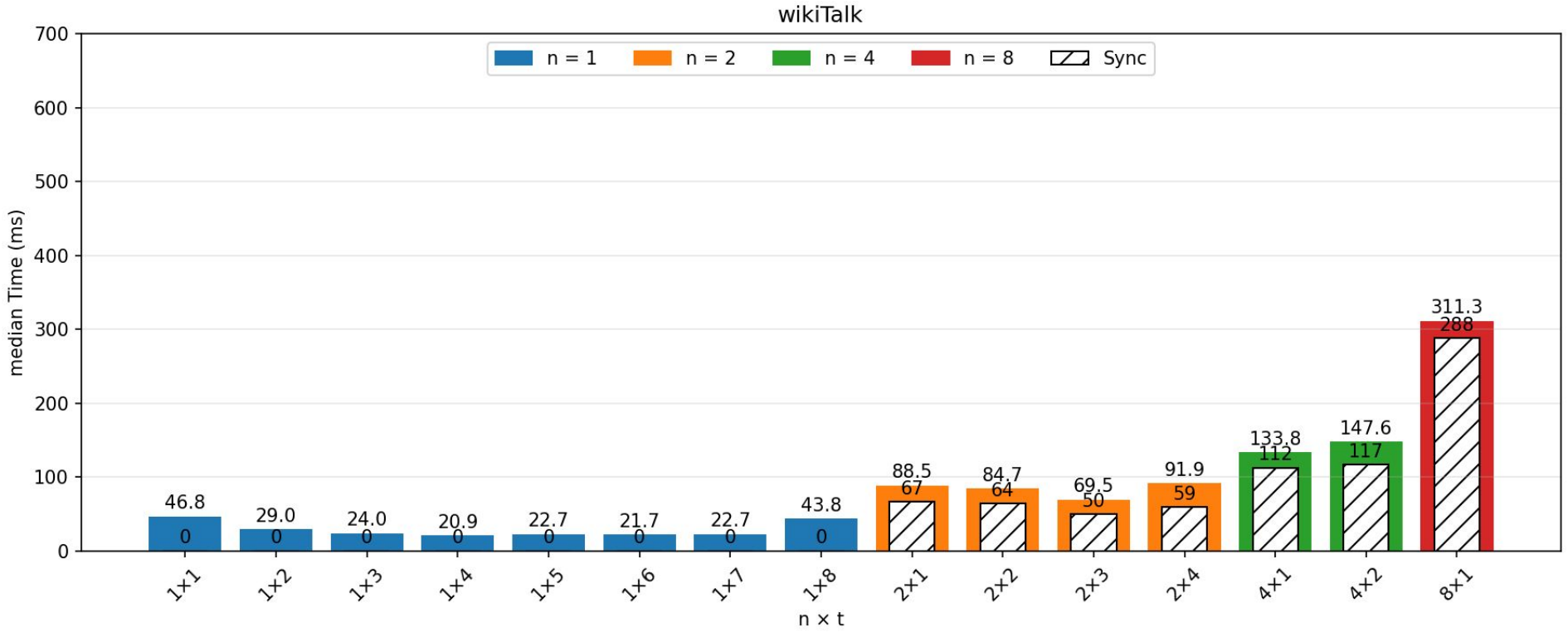
Время работы алгоритма BFS (новое)

web-Google



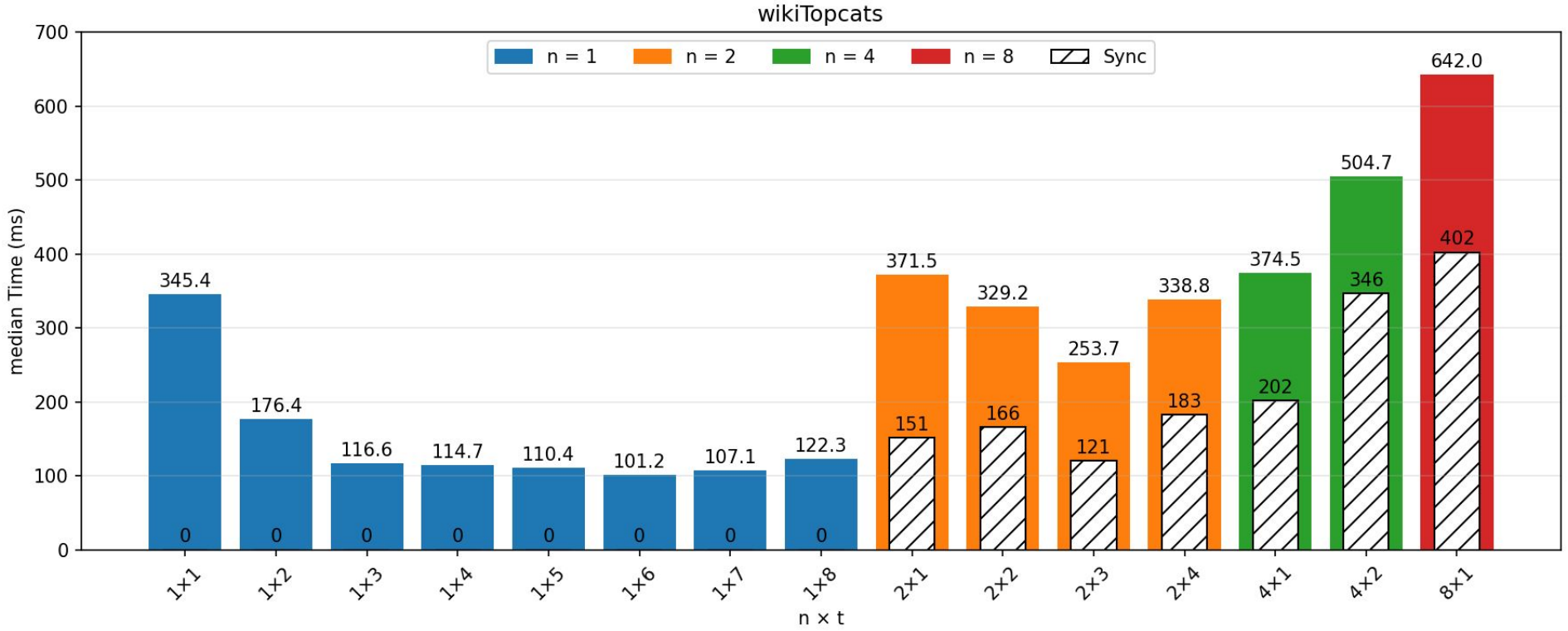
Время работы алгоритма BFS (новое)

wiki-Talk



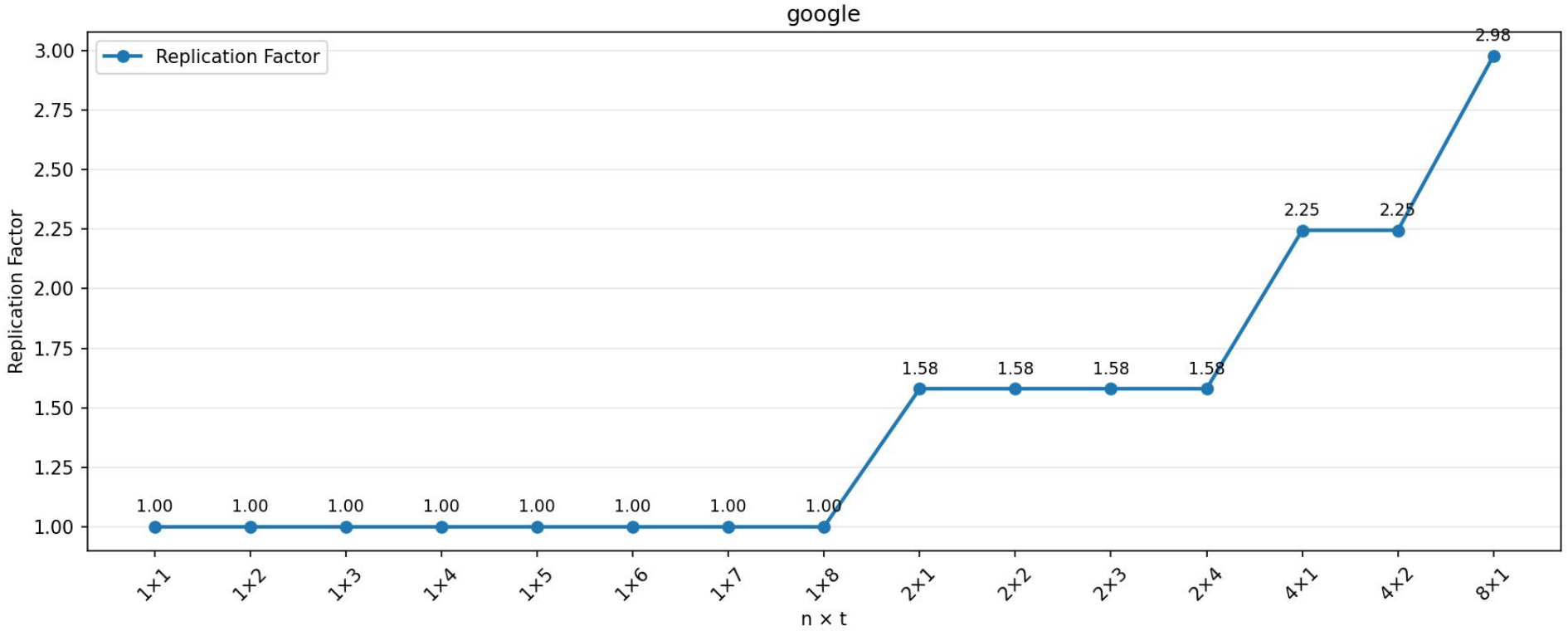
Время работы алгоритма BFS (новое)

wiki-topcats



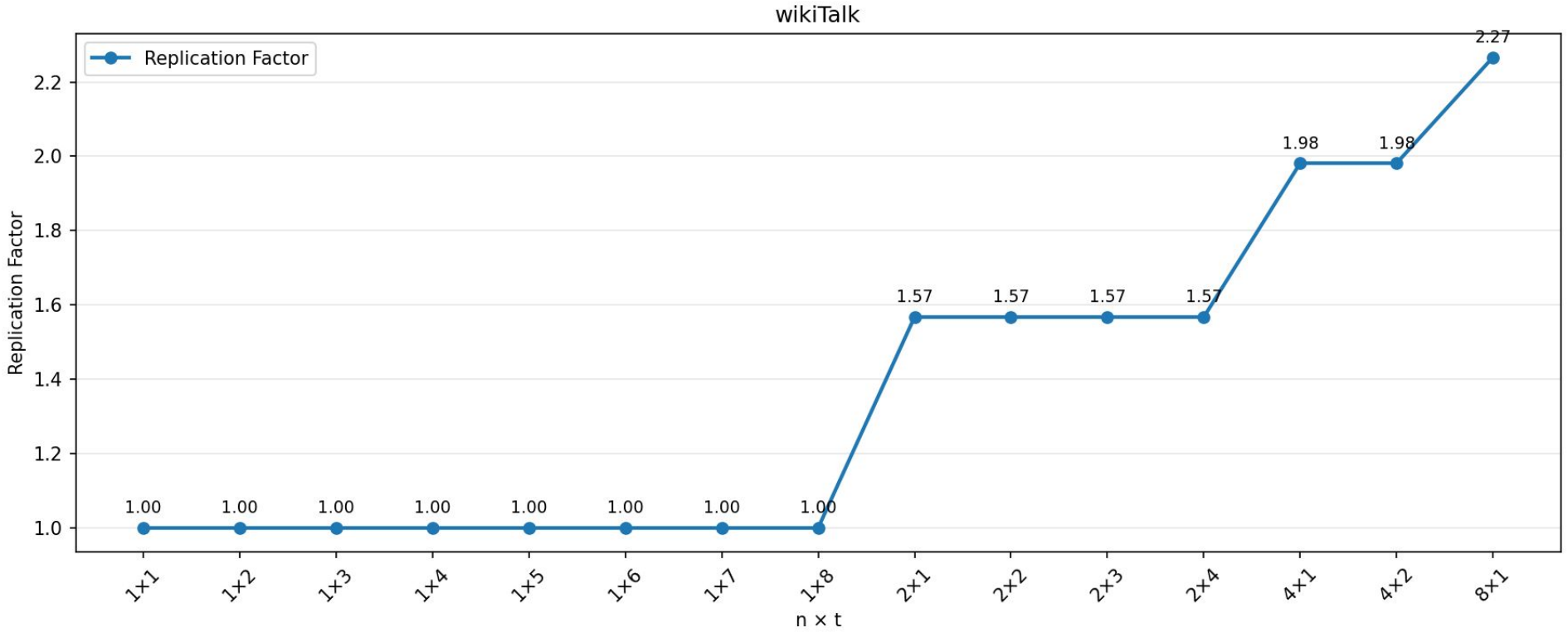
Replication Factor BFS

web-Google



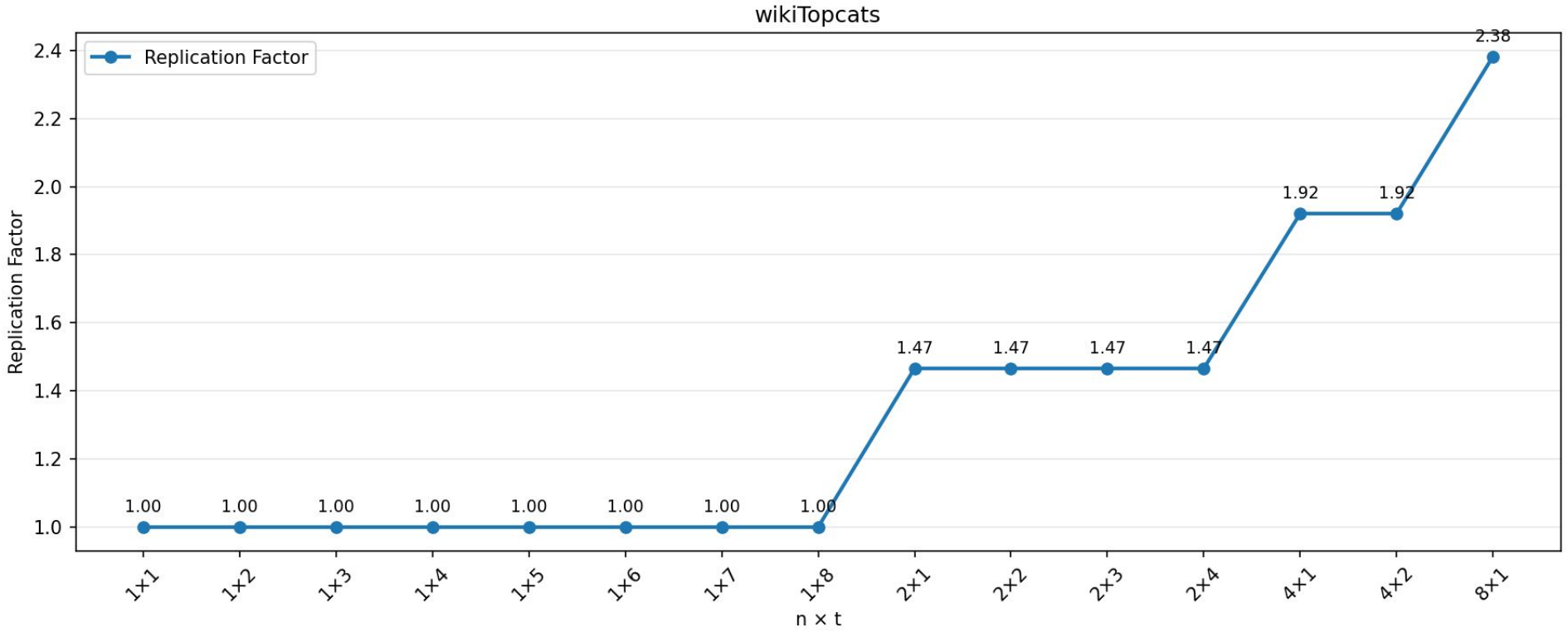
Replication Factor BFS

wiki-Talk



Replication Factor BFS

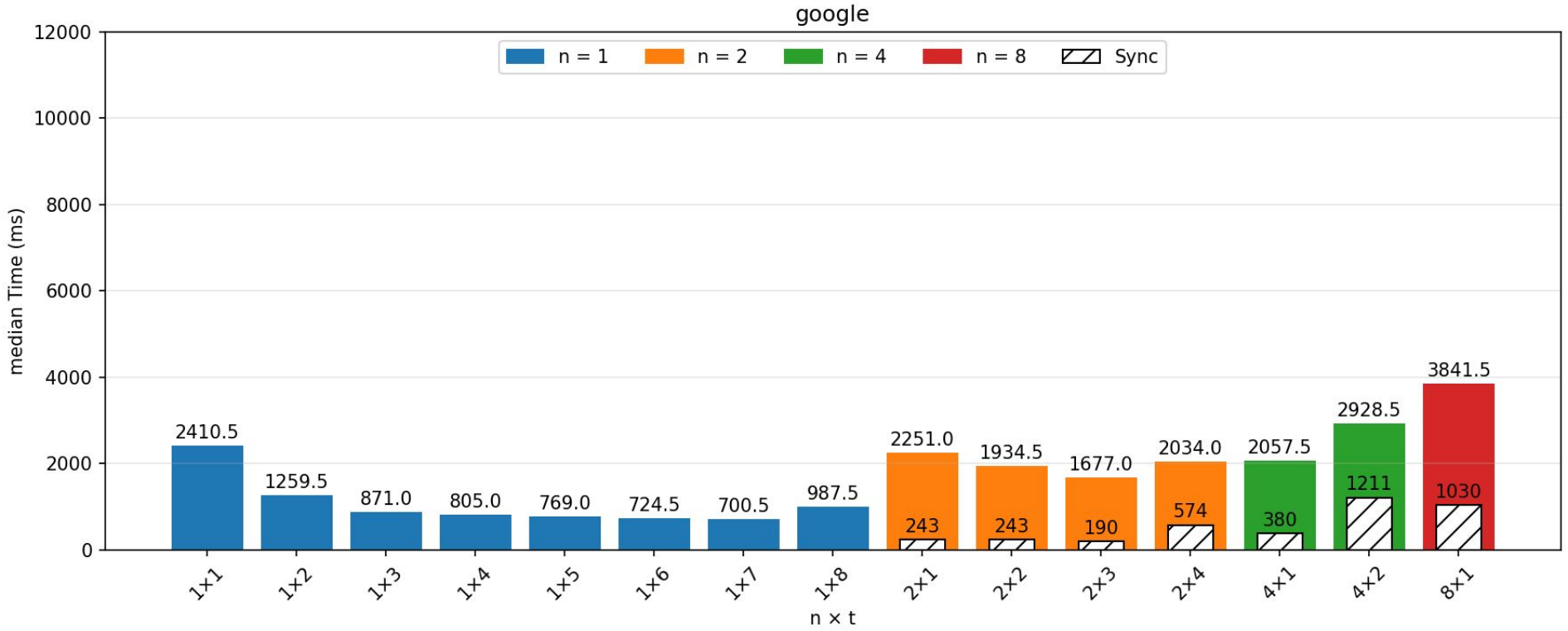
wiki-topcats



Результаты PageRank

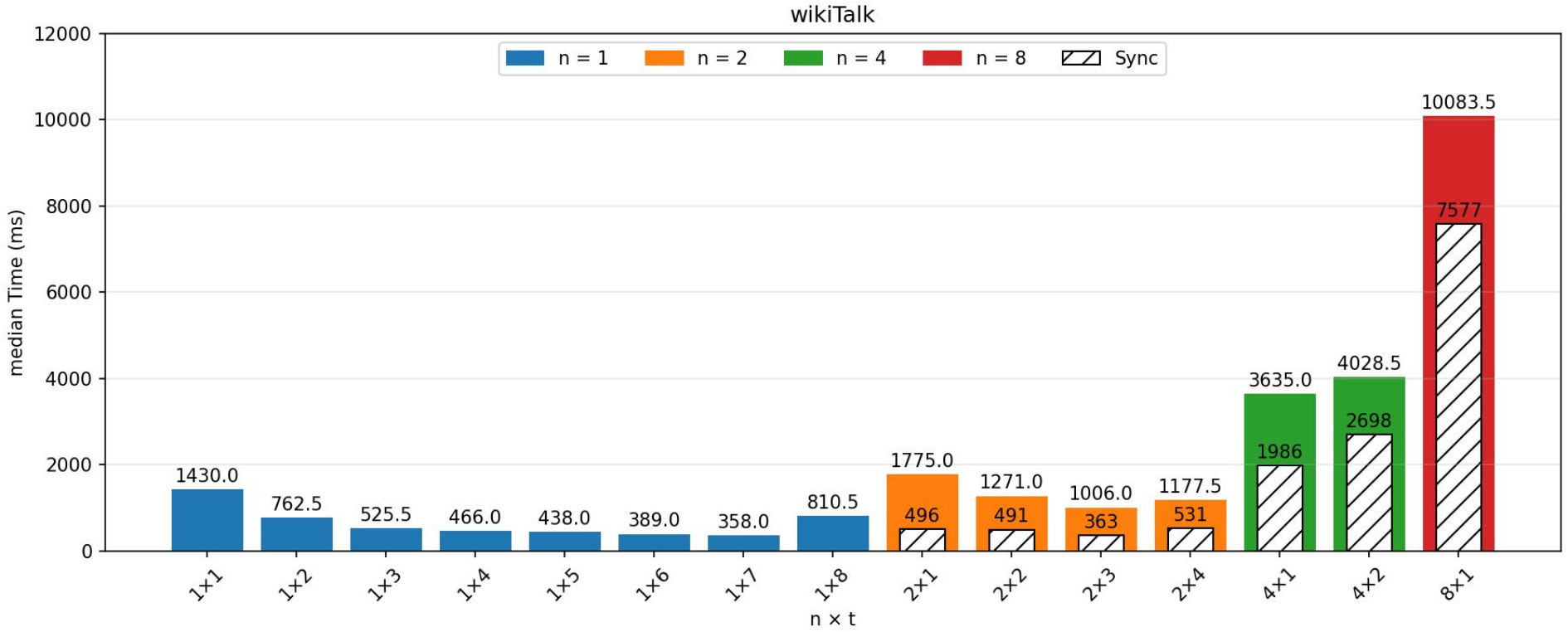
Время работы алгоритма PageRank

web-Google



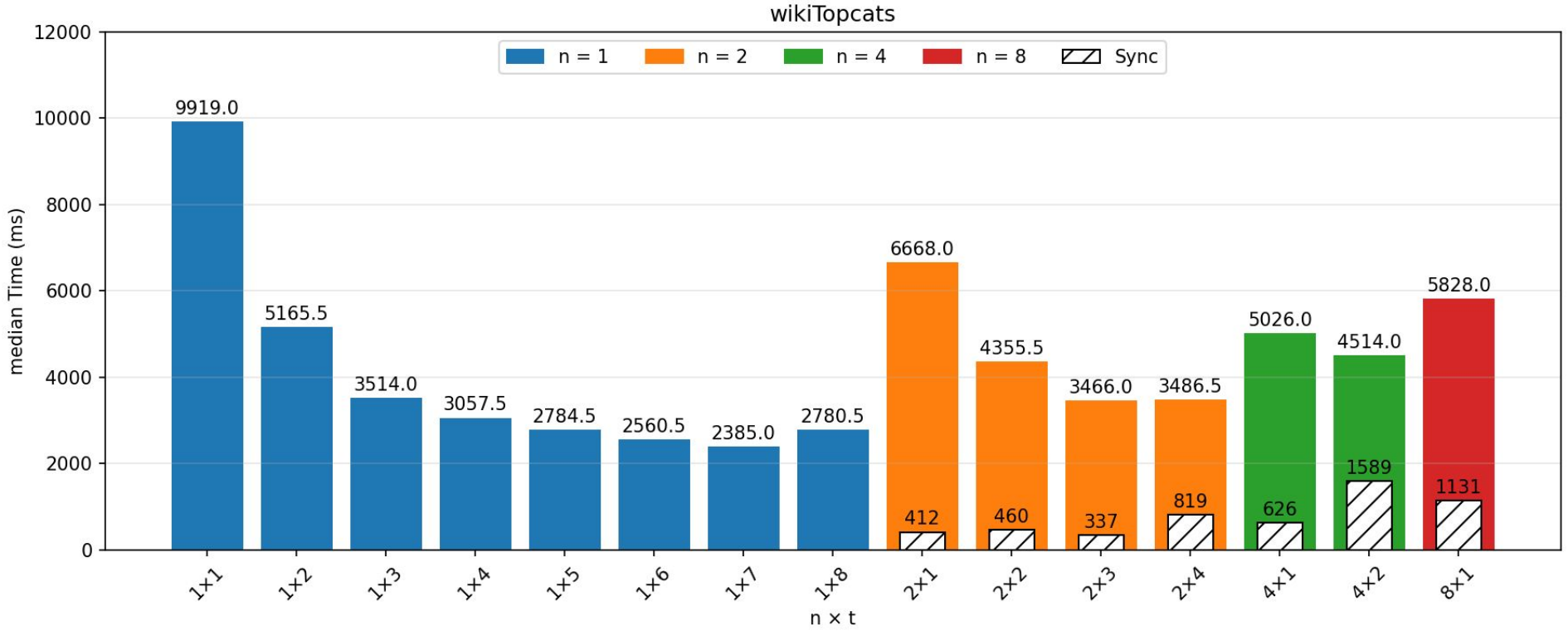
Время работы алгоритма PageRank

wiki-Talk



Время работы алгоритма PageRank

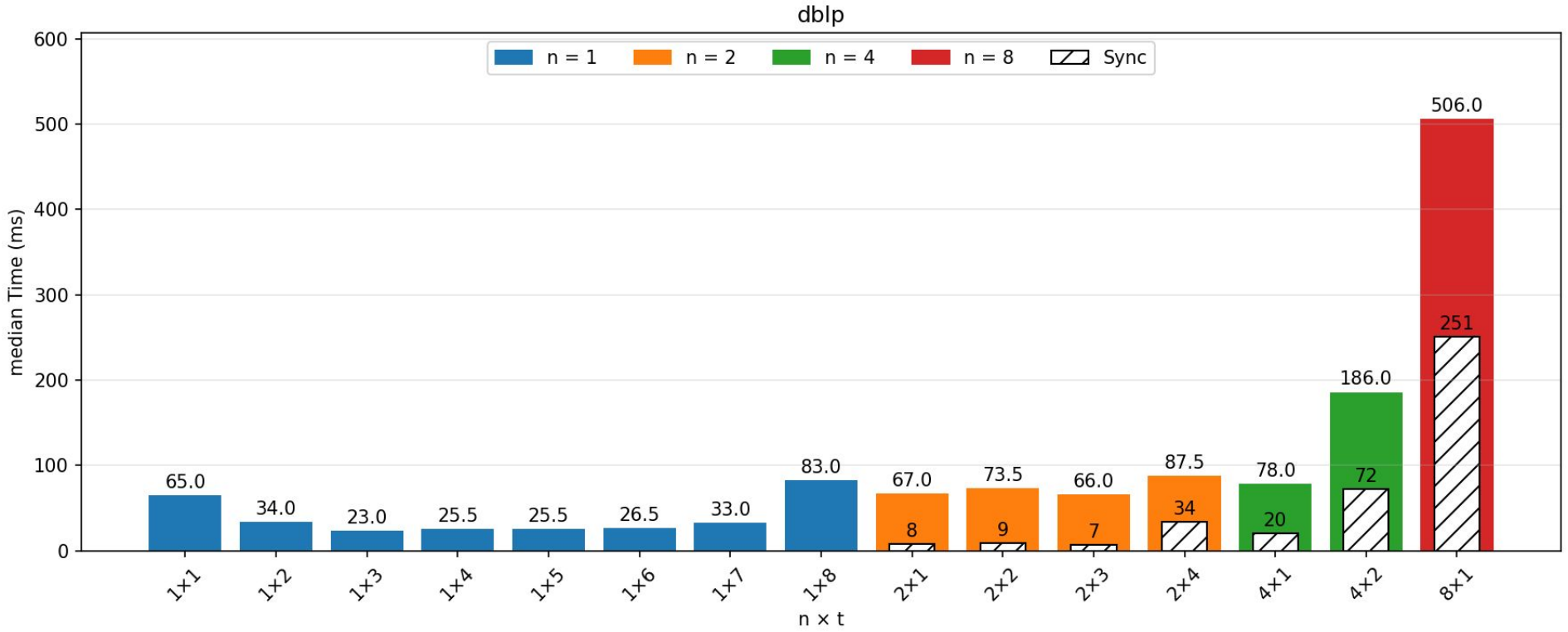
wiki-topcats



Результаты SSSP

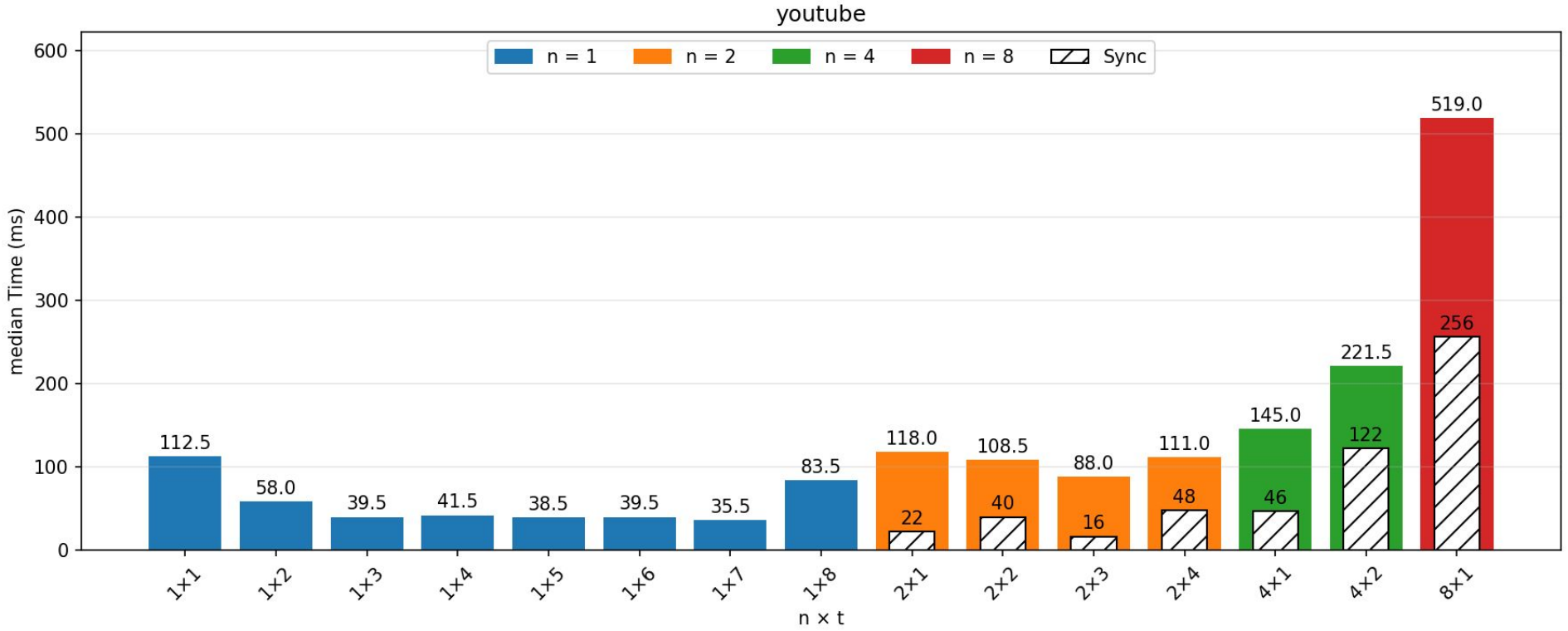
Время работы алгоритма SSSP

com-DBLP



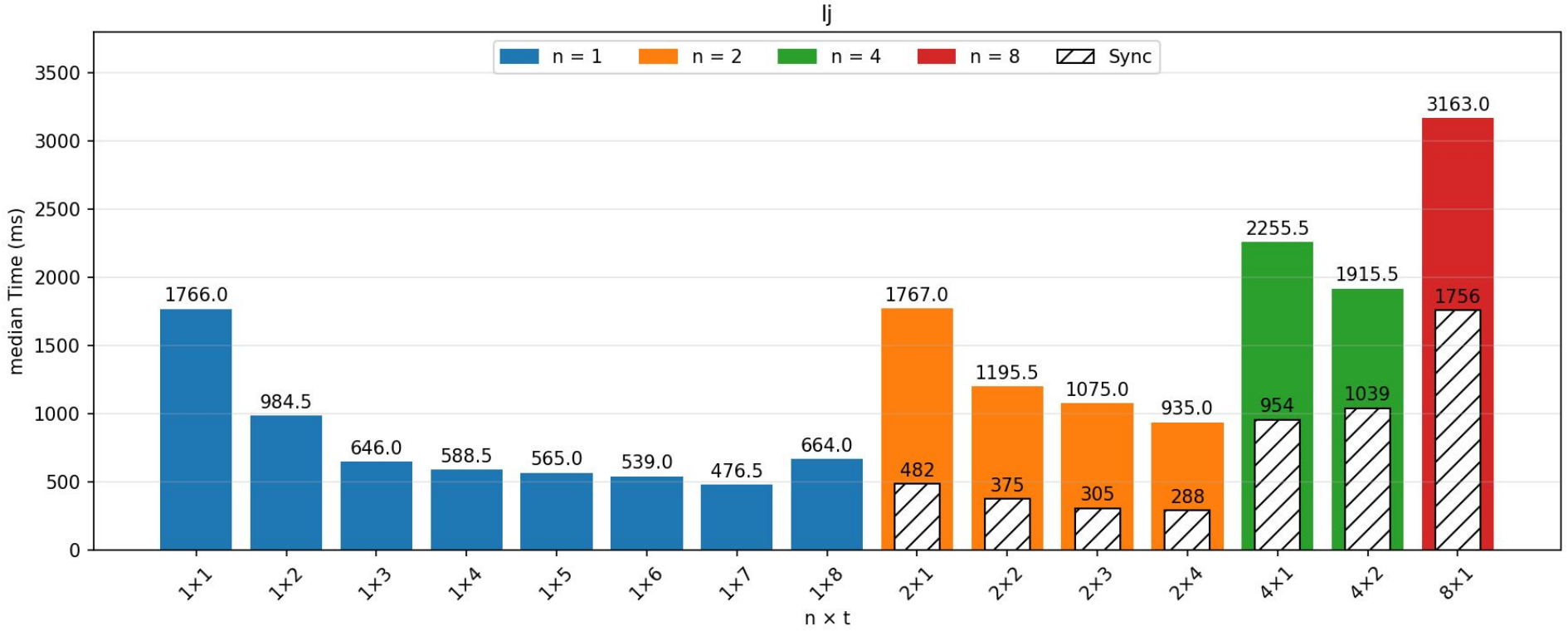
Время работы алгоритма SSSP

com-Youtube



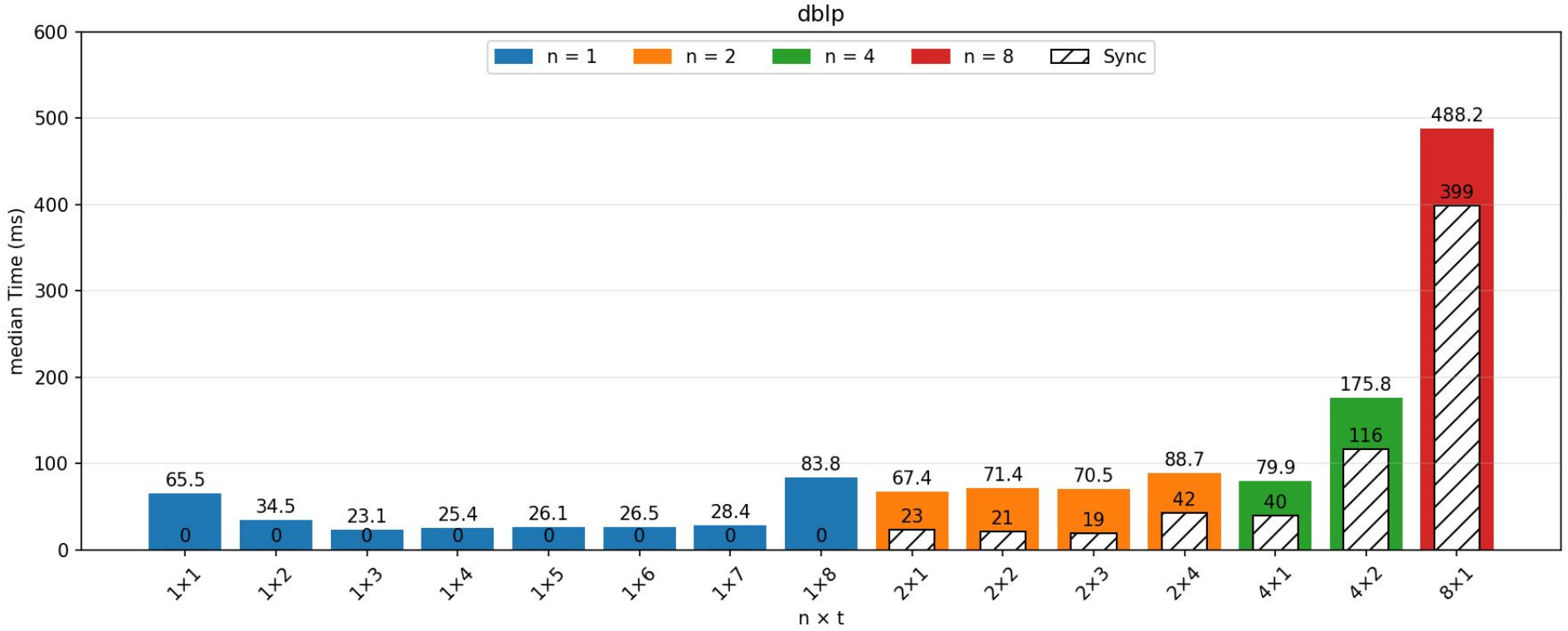
Время работы алгоритма SSSP

com-LiveJournal



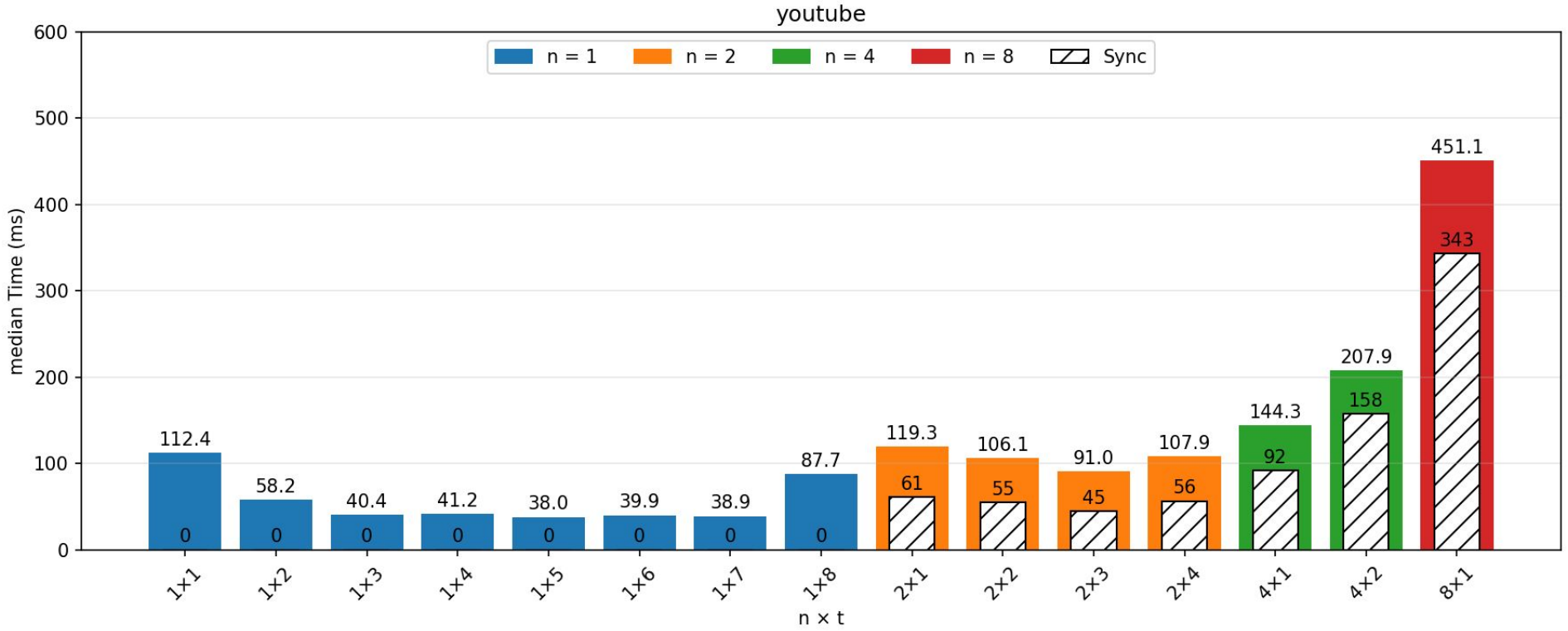
Время работы алгоритма SSSP (новое)

com-DBLP



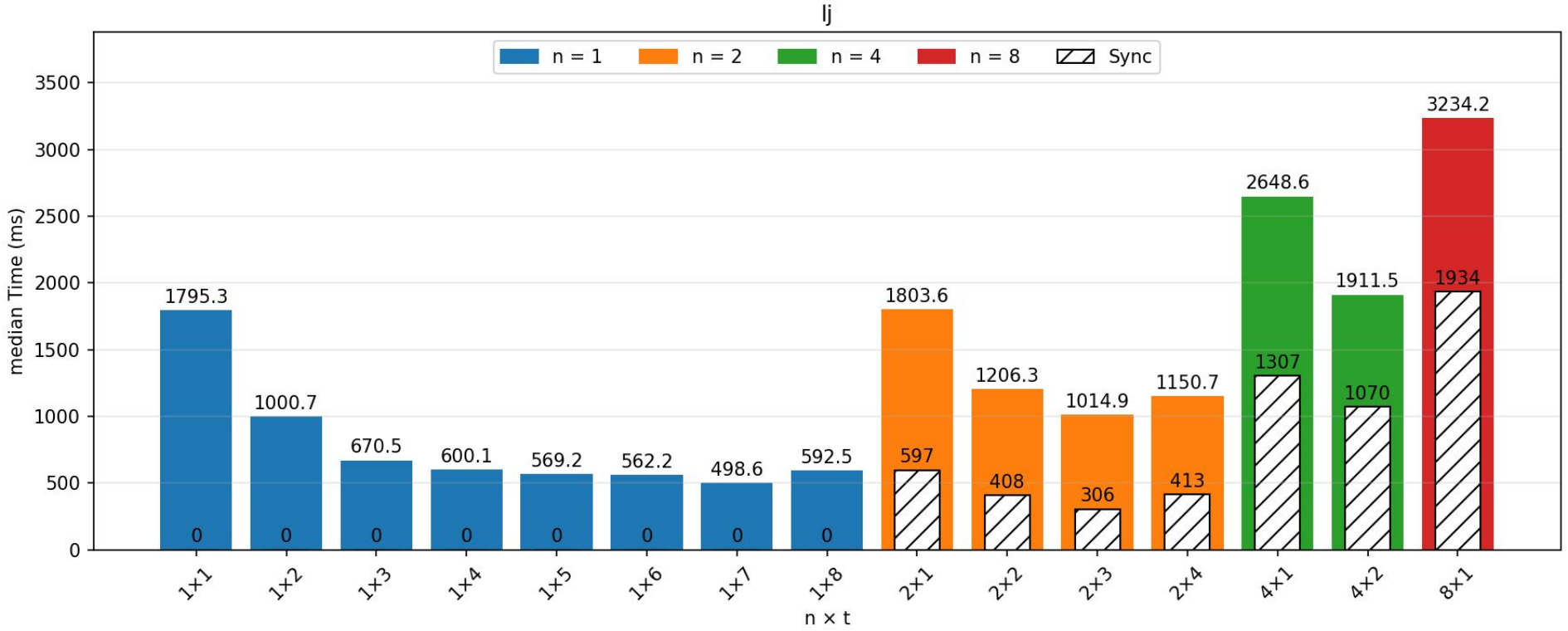
Время работы алгоритма SSSP (новое)

com-Youtube



Время работы алгоритма SSSP (новое)

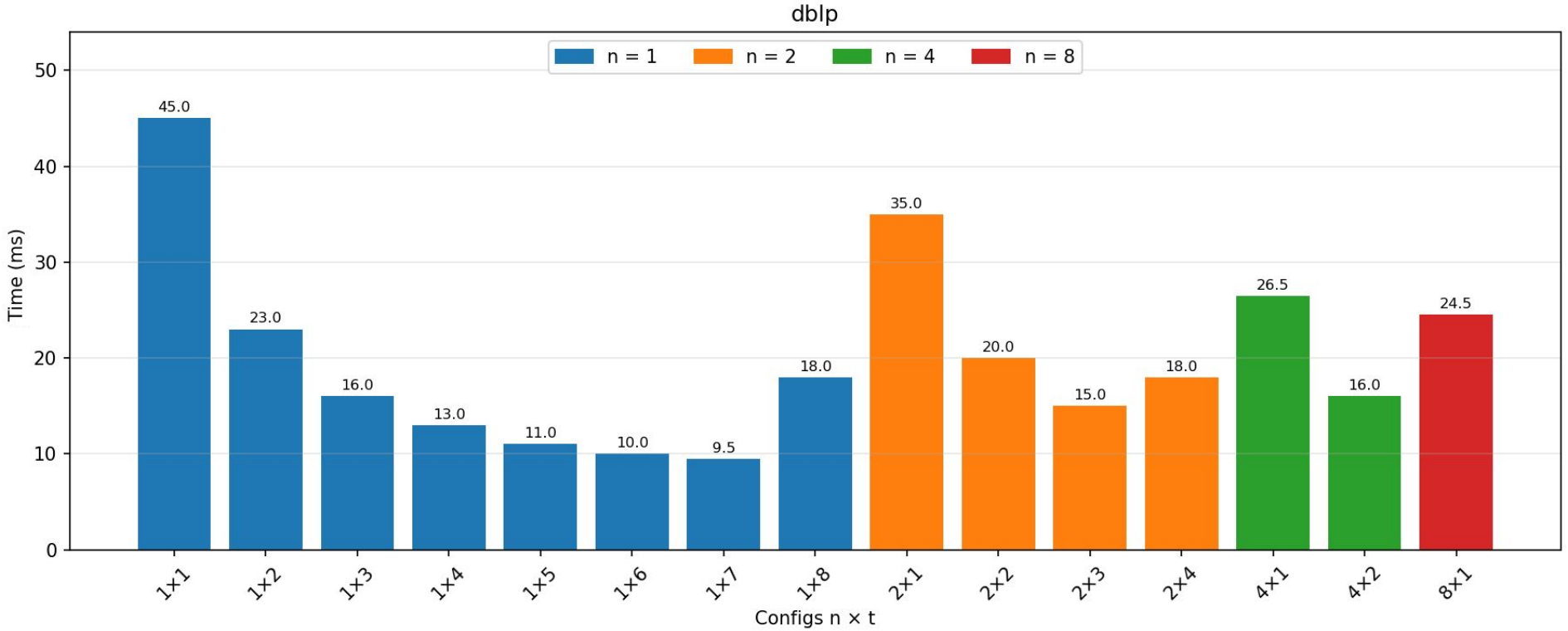
com-LiveJournal



Результаты ТС

Время работы алгоритма TC

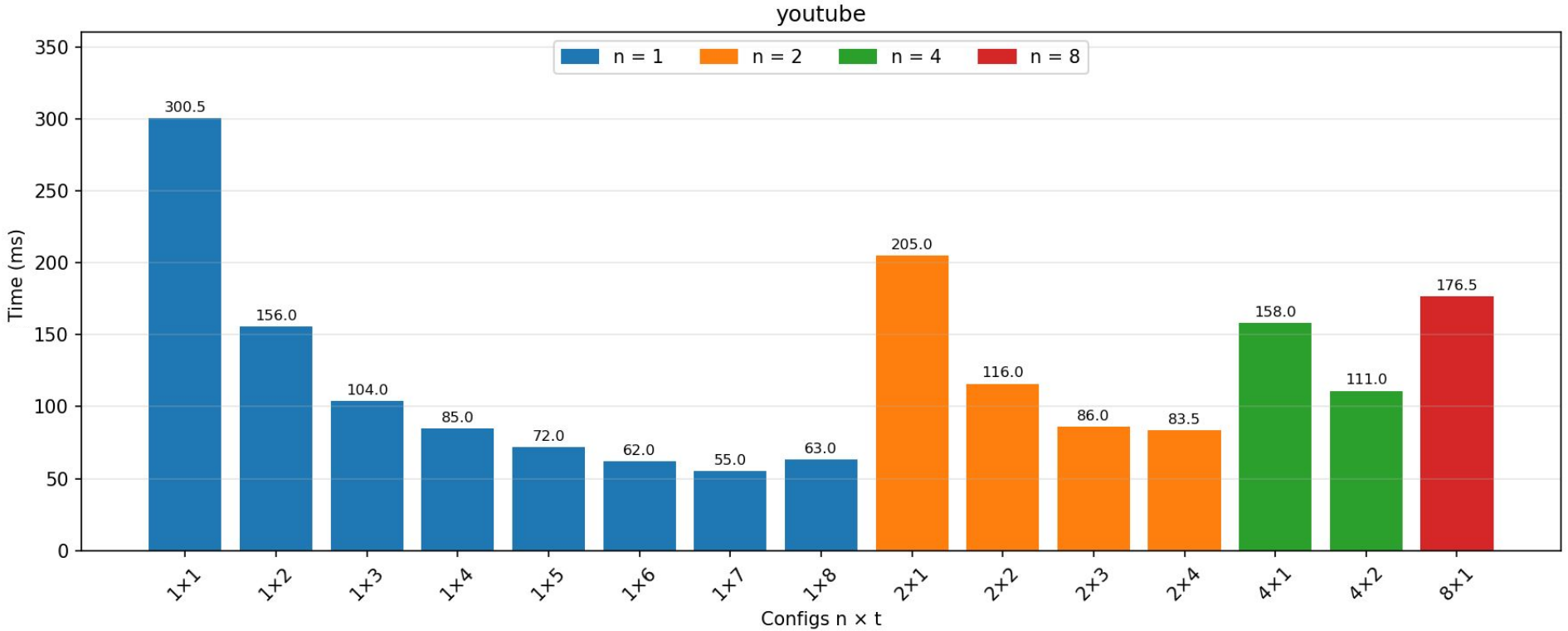
com-DBLP



Алгоритм «free from the synchronization except for the final aggregation»

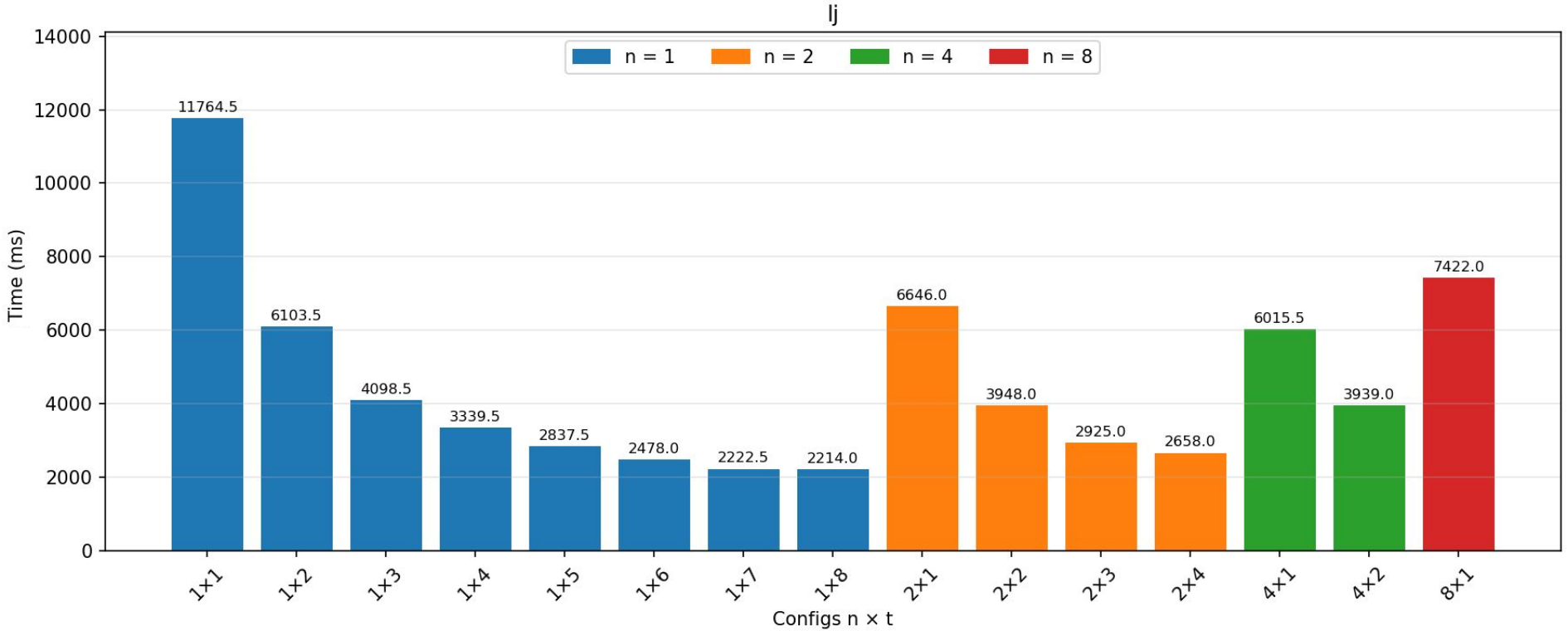
Время работы алгоритма TC

com-Youtube



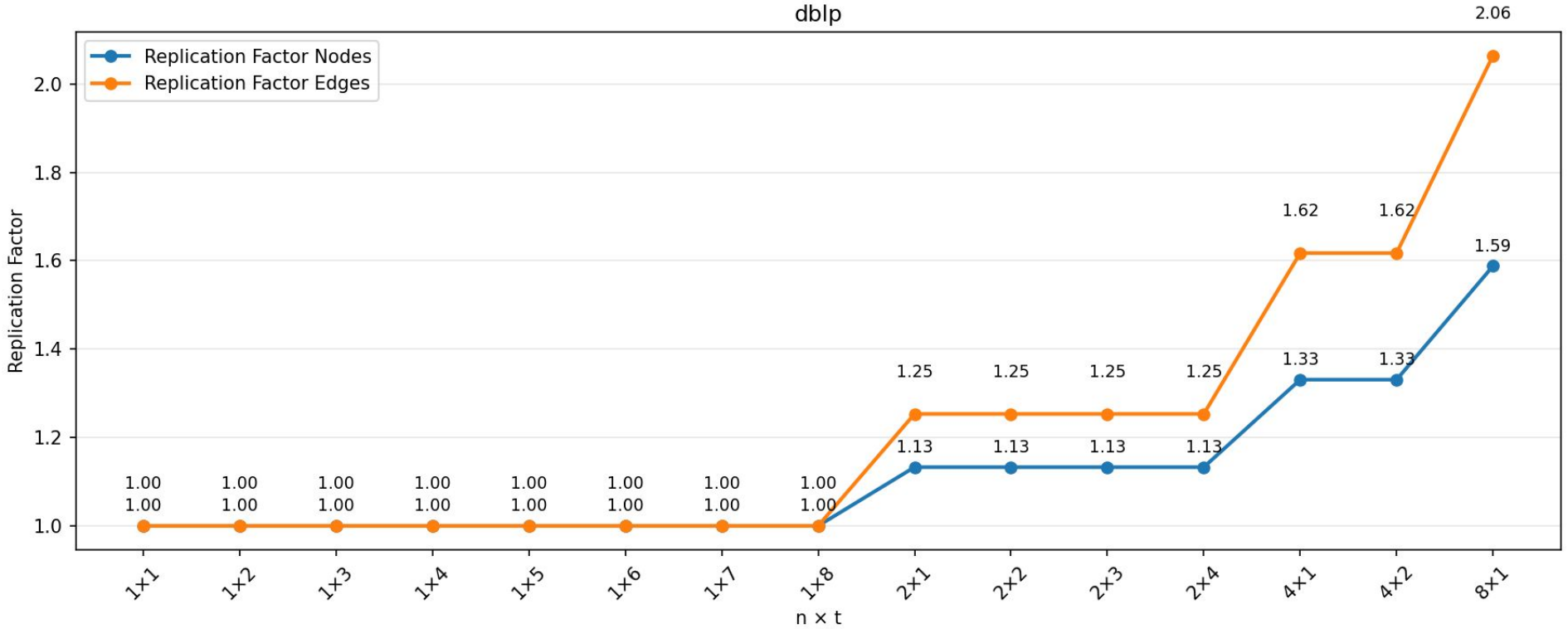
Время работы алгоритма TC

com-LiveJournal



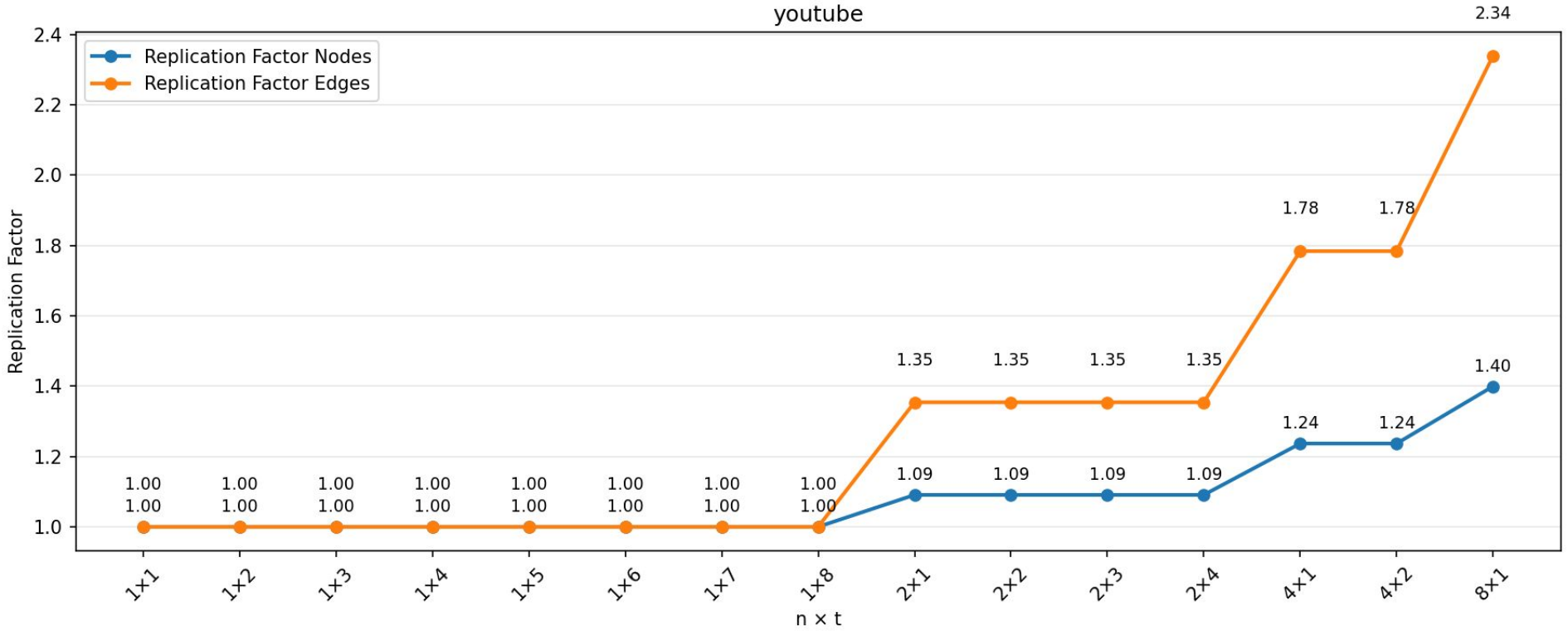
Replication Factor TC

com-DBLP



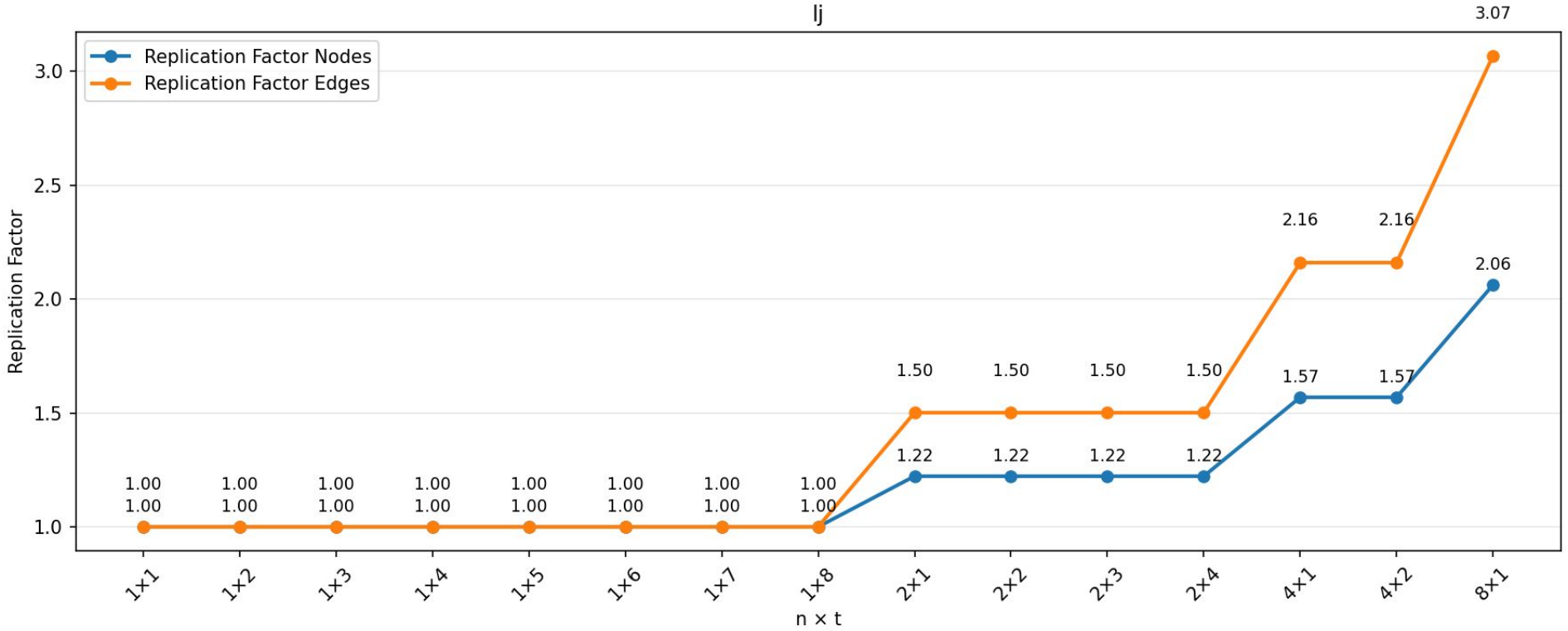
Replication Factor TC

com-Youtube



Replication Factor TC

com-LiveJournal



Вывод

- В условиях этого эксперимента лучше всего производительность у конфигураций с одним процессом и несколькими потоками.
- Распределение по нескольким MPI-процессам дает выигрыш только в случае PageRank (на самом большом графе) и TC, у TC за счет отсутствия затрат на синхронизацию во время работы алгоритма.